

TOETAE SPECIAL

John Derby*

Tue 15/09/2026

Use this documentation as a **guideline** or **reference** of how would you implement the API or request the API. Also for the love of GOD, please ask me if anything written here doesn't make sense.

*Special thanks to Amazon Latte which kept me awake all night

Contents

1	Database Schema	3
2	SqlAlchemy Class	6
2.1	Additional Notes when query assignees	7
3	Ticket Endpoints	8
3.1	GET	8
3.2	POST	9
3.3	POST migrate to PATCH	11
4	Task Endpoints	13
4.1	GET	13
4.2	POST	14
4.3	PATCH	15

1 Database Schema

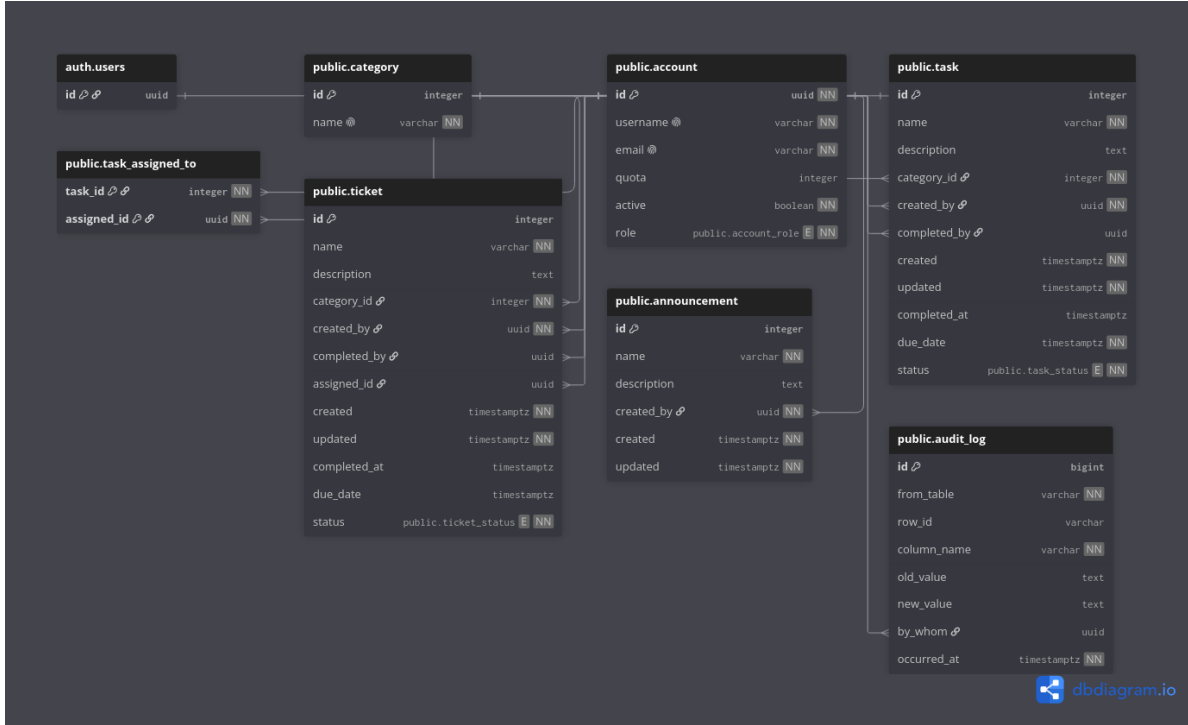


Figure 1: Overview of database schema

Table	Primary key	Foreign keys
account	id (uuid)	id → auth.users.id
category	id (integer)	none
ticket	id (integer)	category_id → category.id; created_by, completed_by, assigned_id → account.id
task	id (integer)	category_id → category.id; created_by, completed_by → account.id
task_assigned_to	(task_id, assigned_id)	task_id → task.id; assigned_id → account.id
announcement	id (integer)	created_by → account.id
audit_log	id (bigint)	by_whom → account.id

Table 1: Tables, keys and relationships

Enum type	Used by	Stored values
account_role	account.role	lab_owner, lab_admin, lab_user
ticket_status	ticket.status	pending, accept, reject
task_status	task.status	in_progress, complete

Table 2: Enum types and the values actually stored in the column

`public.account(id, username, email, quota, active, role)` is the application profile for an authenticated user. `id` is a uuid primary key and foreign key to `auth.users.id`. `username` and `email` are varchar not null. `quota` is an integer. `active` is a boolean not null. `role` is an enum `public.account_role` not null which contains Lab Owner, Lab Admin and Lab user, stored as `lab_owner`, `lab_admin` and `lab_user`.

Additional note. when checking just query the account by `id` and check if `role = lab_user`.

`public.category(id, name)` is a lookup table used to classify tickets and tasks. `id` is an integer primary key and `name` is a varchar not null.

`public.ticket(id, name, description, category_id, created_by, completed_by, assigned_id, created, updated, completed_at, due_date, status)` is a support ticket. `id` is an integer primary key. `category_id` is not null and a foreign key to `category.id`. `created_by` is not null and a foreign key to `account.id`. `completed_by` and `assigned_id` are foreign keys to `account.id`. `created` and `updated` are timestamptz not null. `status` is an enum `public.ticket_status` not null.

Additional note. you don't need to insert/update attributes `created` or `updated` since the database will do that for you automatically.

`public.task(id, name, description, category_id, created_by, completed_by, created, updated, completed_at, due_date, status)` is a unit of work optionally linked to a category. `id` is an integer primary key. `category_id` is not null and a foreign key to `category.id`. `created_by` is not null and a foreign key to `account.id`. `completed_by` is a foreign key to `account.id`. `created` and `updated` are timestamptz not null. `status` is an enum `public.task_status` not null.

Additional note. `date` is the same as `ticket`.

Clarify. `status` in `ticket` could go from Pending → Accept/Reject while `status` in `task` could only go from In-Progress to Complete. They are two separate enum types, not one shared one.

`public.ticket_status` is an enum which contains Pending, Accept and Reject, stored as `pending`, `accept` and `reject`.

`public.task_status` is an enum which contains In-Progress and Complete, stored as `in_progress` and `complete`.

`public.task_assigned_to(task_id, assigned_id)` is a join table assigning accounts to tasks in a many to many relationship. `task_id` is a not null foreign key to `task.id` and `assigned_id` is a not null foreign key to `account.id`, together forming the primary key.

`public.announcement(id, name, description, created_by, created, updated)` is a broadcast message created by an account. `id` is an integer primary key. `created_by` is not null and a foreign key to `account.id`. `created` and `updated` are timestamptz not null.

`public.audit_log(id, from_table, row_id, column_name, old_value, new_value, by_whom, occurred_at)` is a change history recorded across tables. `id` is a bigint primary

key. `from_table` and `column_name` are varchar not null. `row_id` is a varchar. `by_whom` is a foreign key to `account.id`. `occurred_at` is `timestampz` not null.

Additional note. this table is not filled in automatically, there is no trigger behind it, so every endpoint that changes a row has to write its own audit rows.

2 SQLAlchemy Class

Class	Defaults and constraints	Relationships
Category	none	tasks, tickets
Account	role defaults to lab_user; active defaults to true; quota check ≥ 0 ; username and email unique; id cascades on delete	tasks and tickets created, completed and assigned; announcements; audit_entries
Task	status defaults to in_progress; due_date not null; category_id and created_by restrict; completed_by set null	assignees, many to many through task_assigned_to
Ticket	status defaults to pending; due_date optional; category_id and created_by restrict; completed_by and assigned_id set null	none
Announcement	created_by restrict	none
AuditLog	by_whom set null; occurred_at from the database	none

Table 3: ORM classes, every one mapping to the public table of the same name

`Category(id, name)` maps to `public.category`. `tasks` and `tickets` are relationships back to `Task` and `Ticket` through their `category_id`.

`Account(id, username, email, role, quota, active)` maps to `public.account`. `id` is a foreign key to `auth.users.id` with cascade on delete. `username` and `email` are unique and not null. `role` defaults to Lab user. `active` defaults to true. `quota` has a check constraint requiring it to be greater than or equal to zero. It also exposes relationships for tasks and tickets created, completed, assigned, plus `announcements` and `audit_entries`.

`Task(id, name, description, status, category_id, created_by, completed_by, created, updated, completed_at, due_date)` maps to `public.task`. `status` defaults to In-Progress. `category_id` and `created_by` are not null foreign keys with ondelete restrict. `completed_by` is a foreign key with ondelete set null. `created` and `updated` are filled automatically by the database. `due_date` is not null. `assignees` is a many to many relationship to `Account` through the `task_assigned_to` table.

`Ticket(id, name, description, status, category_id, created_by, completed_by, assigned_id, created, updated, completed_at, due_date)` maps to `public.ticket`. `status` defaults to Pending. `category_id` and `created_by` are not null foreign keys with ondelete restrict. `completed_by` and `assigned_id` are foreign keys with ondelete set null. `created` and `updated` are filled automatically by the database. `due_date` is optional.

`Announcement(id, name, description, created_by, created, updated)` maps to `public.announcement`. `created_by` is a not null foreign key with ondelete restrict. `created` and `updated` are filled automatically by the database.

`AuditLog(id, from_table, row_id, column_name, old_value, new_value, by_whom,`

`occurred_at`) maps to `public.audit_log`. `id` is a bigint primary key. `from_table` and `column_name` are not null. `by_whom` is a foreign key with `ondelete` set `null`. `occurred_at` is filled automatically by the database.

`TicketStatus`, `TaskStatus` and `AccountRole` are Python string enums that back the `ticket_status`, `task_status` and `account_role` Postgres enum types, storing lowercase values such as `in_progress` rather than the member names.

2.1 Additional Notes when query assignees

Since `Task.assignees` is a collection (many to many through `task_assigned_to`), you have to eager load it with `selectinload`. On an async session the default lazy load cannot fire on attribute access, so reading `task.assignees` without it raises `MissingGreenlet`. `selectinload` also leaves the parent row count alone, unlike `joinedload`, which repeats the `Task` row once per assignee.

```
from sqlalchemy import select
from sqlalchemy.orm import selectinload

task = await session.scalar(
    select(Task).where(Task.id == task_id)
    .options(selectinload(Task.assignees))
)
```

That emits two queries, the second one loading the collection:

```
SELECT ... FROM task WHERE task.id = 5;

SELECT ... FROM account
JOIN task_assigned_to
  ON account.id = task_assigned_to.assigned_id
WHERE task_assigned_to.task_id IN (5);
```

Without `.options(selectinload(Task.assignees))` there is no second query at all, and touching `task.assignees` raises `MissingGreenlet`.

Read more: <https://docs.sqlalchemy.org/en/20/orm/queryguide/relationships.html#sqlalchemy.orm.selectinload>

3 Ticket Endpoints

3.1 GET

GET /tickets lists tickets. Every parameter is optional.

Scope comes first. An admin, meaning `lab_admin` or `lab_owner`, sees every ticket. Everyone else sees only their own, the rows where `created_by` is their account id. Resolve the role with the account lookup from section 1, then AND that filter onto every query, never OR, and never let it come from a parameter. No parameter can widen it, which is why `created_by` is not one of them.

```
SELECT * FROM ticket
WHERE ((:is_admin AND NOT :only_owned)
      OR created_by = :caller_id)
      AND (:status IS NULL OR status = :status)
ORDER BY created DESC
LIMIT :limit
```

Parameters, all optional:

```
GET /tickets
?ticket_id=41
&status=pending
&category_id=2
&due_before=2026-10-01T00:00:00Z
&only_owned=true
&limit=20
```

There are six and no more: `ticket_id`, `status`, `category_id`, `due_before`, `only_owned` and `limit`.

`ticket_id` narrows to a single ticket and still goes through the scope filter, so a `lab_user` naming somebody else's ticket gets an empty page rather than that ticket. `status` takes the stored lowercase value. `category_id` filters on the column directly. `due_before` is an upper bound on `due_date`, and a ticket with a null due date falls out of the results as soon as it is used. `only_owned` is a boolean defaulting to false. It lets an admin narrow the page to the tickets they created themselves, instead of the whole table they would otherwise get. For a `lab_user` it changes nothing, they were already limited to their own, so it is accepted and quietly does nothing rather than being an error. Note which direction it runs, it only ever narrows the scope and never widens it, which is what keeps it safe to take from a client at all. `limit` defaults to 20 and caps at 100, and cap it rather than trusting it. There is no sort parameter, the order is fixed at newest first.

An unknown parameter is an error, not something to ignore, since a dropped filter looks like a filter that matched everything.

Response 200 OK. `total` is the count before paging:

```
{
  "items": [
    {
      "id": 41,
      "name": "Projector in Lab 3 wont turn on",
      "description": "No power LED",
      "category_id": 2,
      "created_by": "9f1c...e30a",
      "completed_by": null,
      "assigned_id": null,
      "status": "pending",
      "created": "2026-09-16T09:12:44.512Z",
      "updated": "2026-09-16T09:12:44.512Z",
      "completed_at": null,
      "due_date": "2026-09-30T17:00:00Z"
    }
  ],
  "total": 37,
  "limit": 20
}
```

No matches is an empty `items` with `total` zero, not a 404.

GET `/tickets/{id}` returns one ticket without the wrapper, same scope rule. When a `lab_user` asks for a ticket that exists but is not theirs, answer 404, not 403. A 403 tells them it exists.

Reading writes nothing, no audit row, no quota, no `updated` bump.

Failure cases: 401 for a bad token, 403 when `active` is false, 404 on a single ticket that does not exist or is out of scope, 422 for a bad enum value, a bad number, a bad timestamp or an unknown parameter, 500 otherwise without leaking the database error.

Additional note. `own` is written here as `created_by` only. If a ticket can be assigned to a `lab_user`, make it `created_by = :caller_id` OR `assigned_id = :caller_id`, or somebody cannot see work assigned to them.

3.2 POST

POST `/tickets` creates a ticket. Requires a bearer token, and `created_by` always comes from that token, never from the body.

```
{
  "name": "Projector in Lab 3 wont turn on",
  "description": "No power LED, tried two outlets",
  "category_id": 2,
  "assigned_id": null,
  "due_date": "2026-09-30T17:00:00Z"
}
```

`name` and `category_id` are required, the rest are optional. `category_id` and `assigned_id` must resolve to existing rows, so check them yourself and give the caller a readable error instead of a foreign key violation. `due_date` is ISO 8601 with an offset.

Everything else is server side and a client may not send it. `id` is generated, `created_by` comes from the token, `status` is always Pending on create, `created` and `updated` are filled by the database, `completed_by` and `completed_at` stay null.

Quota. `account.quota` is a nullable integer spent one per ticket. Null means no allowance configured, so unlimited, deduct nothing. Above zero is spent by deducting one. Zero means refused and no ticket row is written. Deduct with a conditional update rather than reading the value and writing it back, otherwise two requests arriving together both read the same number and drive the column to -1, which the check constraint rejects:

```
UPDATE account
  SET quota = quota - 1
 WHERE id = :caller_id
   AND quota > 0
RETURNING quota;
```

A row back means the deduction took, nothing back means the quota was already zero. A deduction is not refunded when a ticket is later rejected, so quota counts tickets opened rather than tickets outstanding.

Audit. Nothing writes `audit_log` for you, there is no trigger, so a change is recorded only if the endpoint inserts it. The table is column oriented, `column_name` is not null, so one row is one changed column. `row_id` is a varchar and not a real foreign key, which is what lets it hold an integer `ticket.id` and a uuid `account.id`, so stringify whichever key you mean. `old_value` and `new_value` are text. `by_whom` is the caller. `occurred_at` is filled by the database.

```
INSERT INTO audit_log
  (from_table, row_id, column_name,
   old_value, new_value, by_whom)
VALUES
  ('ticket', '41', 'status',
   NULL, 'pending', :caller_id),
  ('account', :caller_id, 'quota',
   '5', '4', :caller_id);
```

The ticket insert, the quota update and the audit rows go in one transaction, they commit together or not at all.

Response 201 **Created** returns the new row in the same shape as GET, with `status` as the stored lowercase `pending`.

Failure cases: 401 for a bad token, 403 when `active` is false or the role may not set a field that was sent, 403 with the body below when the quota is exhausted, 422 when `name` or `category_id` is missing, a timestamp will not parse, a foreign key does not resolve or a server side field was supplied, 500 otherwise without leaking the database error.

```
{
  "detail": "Ticket quota exhausted",
  "code": "quota_exhausted",
  "quota": 0
}
```

Additional note. a `lab_user` raising their own ticket probably should not choose `assigned_id`, while an admin should. If you want that rule, check the role and drop the field for a `lab_user`.

3.3 POST migrate to PATCH

PATCH `/tickets/{id}` updates part of a ticket. Same auth as POST.

Additional note. there is no PUT here and that is deliberate. PUT means replace the whole row, so a client sending only `status` would null out the description and the due date along with it. Updates on a ticket are partial, which is what PATCH means. If you were expecting PUT in the route table it is not missing, it is this.

Only the fields being changed go in the body. A field left out is untouched, so accepting a ticket is a body with `status` in it and nothing else. An empty body is a bad request rather than a successful no-op.

```
{
  "assigned_id": "4b7d...91cc",
  "status": "accept"
}
```

Left out and sent as null are different instructions. Out means do not touch. Null means clear, and that only works on `description`, `assigned_id` and `due_date`, the nullable columns. Null for `name`, `category_id` or `status` is a validation error.

`id`, `created_by` and `created` are immutable, so reject an attempt to change them rather than ignoring it. `updated` is refreshed by the database. `completed_by` and `completed_at` are derived, never taken from the body.

Status goes from Pending to Accept or Reject and that is the only move there is. Both are terminal, an accepted ticket cannot later be rejected and a rejected one cannot be re-opened. Anything else is refused. Editing a Pending ticket while leaving `status` as Pending is just an edit.

When the status leaves Pending, set `completed_by` to the caller and `completed_at` to now. A rejected ticket counts as completed, the column records who closed it, not who agreed with it.

Guard the transition with the old status in the where clause, the same way the quota deduction is guarded, so two admins pressing accept and reject at the same moment cannot both win:

```

UPDATE ticket
  SET status = 'accept',
      completed_by = :caller_id,
      completed_at = now()
  WHERE id = :ticket_id
      AND status = 'pending'
  RETURNING id;

```

No row back means somebody already closed it, so return the conflict rather than reporting success.

Quota is untouched here, it is spent on open and not refunded on reject.

The audit matters more here than on POST because now there are real previous values. Write one row per column that actually changed, with both `old_value` and `new_value` filled. A field that was sent but already held that value is not a change, do not write a row for it.

```

INSERT INTO audit_log
  (from_table, row_id, column_name,
   old_value, new_value, by_whom)
VALUES
  ('ticket', '41', 'status',
   'pending', 'accept', :caller_id),
  ('ticket', '41', 'assigned_id',
   NULL, '4b7d...91cc', :caller_id);

```

Same transaction as the update, and again nothing writes them for you.

Response 200 OK returns the updated row in the same shape as GET.

Failure cases: 401 for a bad token, 403 when `active` is false or the role may not make this change, 404 when no ticket has that id, 409 with the body below when the transition is not legal or another request closed the ticket first, 422 for the POST body problems plus an empty body, a null on a not null column, or an attempt to write an immutable or derived field, 500 otherwise.

```

{
  "detail": "Ticket is already accept",
  "code": "invalid_status_transition",
  "from": "accept",
  "to": "reject"
}

```

Additional note on roles. the rule I would expect is a `lab_user` editing the wording of their own Pending ticket and nothing else, with `accept`, `reject` and `assign` belonging to `lab_admin` and `lab_owner`, and nobody accepting a ticket they opened themselves. That is policy rather than schema, so confirm it.

4 Task Endpoints

Tasks work like tickets, so this section covers only what is different. Auth, the error bodies, the one transaction rule and the audit mechanics are all the same as section 3.

Three differences drive the rest. A task has no `assigned_id` column, assignment lives in the many to many `task_assigned_to`, so assignees are a list rather than a field. `due_date` is not null on a task, where on a ticket it was optional. And `task_status` only goes `in_progress` $\rightarrow$ `complete`, there is no pending, accept or reject, so there is nothing to approve here, only work to finish.

4.1 GET

GET `/tasks` lists tasks. Every parameter is optional.

Scope is wider than it was for tickets. An admin sees every task. Everyone else sees the tasks they created and the tasks they are assigned, and that second half is the whole reason the join table exists, so it cannot be left out:

```
SELECT t.* FROM task t
  WHERE ((:is_admin AND NOT :only_owned)
         OR t.created_by = :caller_id
         OR EXISTS (SELECT 1
                    FROM task_assigned_to a
                    WHERE a.task_id = t.id
                        AND a.assigned_id = :caller_id))
 ORDER BY t.created DESC
LIMIT :limit
```

The parameters are the six from section 3.1 with `task_id` in place of `ticket_id`, and `status` here takes `in_progress` or `complete`. `only_owned` means the same thing but covers both halves of the wider scope, so an admin setting it gets the tasks they created and the tasks they are assigned, not just the ones they created.

Each task comes back with an `assignees` array of account ids, which is the one field tickets do not have:

```
{
  "id": 88,
  "name": "Re-image the lab 2 machines",
  "description": "Ubuntu 24.04, standard image",
  "category_id": 3,
  "created_by": "9f1c...e30a",
  "completed_by": null,
  "assignees": ["4b7d...91cc", "7a02...b511"],
  "status": "in_progress",
  "created": "2026-09-16T09:12:44.512Z",
  "updated": "2026-09-16T09:12:44.512Z",
  "completed_at": null,
  "due_date": "2026-09-30T17:00:00Z"
}
```

Load that array with `selectinload`, as section 2.1 says. On a list endpoint it costs one extra query for the whole page rather than one per task, and it is not optional on an async session, a lazy load there raises `MissingGreenlet` rather than quietly working.

4.2 POST

`POST /tasks` creates a task. `created_by` comes from the token as always.

```
{
  "name": "Re-image the lab 2 machines",
  "description": "Ubuntu 24.04, standard image",
  "category_id": 3,
  "due_date": "2026-09-30T17:00:00Z",
  "assignees": ["4b7d...91cc", "7a02...b511"]
}
```

`name`, `category_id` and `due_date` are all required. The required due date is the difference from `POST` on a ticket, the column is not null so there is no creating a task without one.

`assignees` is a list of account ids and may be empty or absent, which creates an unsigned task. Every id in it must exist, so check them in one query rather than one per id, and reject the whole request if any fails rather than creating the task with the assignees that happened to resolve. Repeated ids are not an error but they are also not two rows, the pair is the primary key of `task_assigned_to`, so collapse them before inserting or the insert fails on the duplicate.

`status` is always `in_progress` on create, a client cannot open a task that is already complete. `completed_by` and `completed_at` stay null.

Quota is not touched here. It is spent per ticket, and nothing in section 1 ties it to tasks. If tasks are meant to cost quota as well then say so and it becomes the same conditional update as in section 3.2, but as written a task is free.

The audit needs a row per populated task column, then one more per assignee, and those point at the join table rather than at the task:

```

INSERT INTO audit_log
  (from_table, row_id, column_name,
   old_value, new_value, by_whom)
VALUES
  ('task', '88', 'status',
   NULL, 'in_progress', :caller_id),
  ('task_assigned_to', '88', 'assigned_id',
   NULL, '4b7d...91cc', :caller_id);

```

`row_id` here is the task id, since that is what identifies the row people will search for later. The task insert, the `task_assigned_to` inserts and the audit rows are one transaction.

Response 201 `Created` returns the task in the shape above, assignees included.

Failure cases are those of section 3.2, minus the quota case, plus 422 when `due_date` is missing or when an id in `assignees` does not resolve to an account.

4.3 PATCH

`PATCH /tasks/{id}` updates part of a task. Partial body, same omitted versus null rule as section 3.3.

`due_date` is the exception to that rule. It can be changed but it cannot be cleared, because the column is not null, so a null there is a validation error rather than a way to remove a deadline.

Status goes `in_progress` → `complete` and that is the only move. Complete is terminal, a finished task does not go back to in progress. Guard it with the old status in the where clause exactly as section 3.3 does, and set `completed_by` and `completed_at` when the status leaves `in_progress`:

```

UPDATE task
  SET status = 'complete',
      completed_by = :caller_id,
      completed_at = now()
  WHERE id = :task_id
     AND status = 'in_progress'
RETURNING id;

```

`assignees` is the awkward one, because a list has no partial form. Leaving it out means do not touch the assignments. Sending it means replace the whole set with what was sent, so sending an empty list unassigns everybody. There is no way to add one person without naming the others, and if you want that it needs its own endpoint rather than a special case here.

That replacement is a set difference, not a delete and reinsert. Work out which ids were added and which were removed, apply only those, and leave the unchanged rows alone. Reinserting a row that was already there gives you a primary key violation and, worse, an audit trail saying somebody was unassigned and reassigned when nothing actually happened to them.

Audit one row per added or removed assignee, with the direction carried by which side is null:

```
INSERT INTO audit_log
  (from_table, row_id, column_name,
   old_value, new_value, by_whom)
VALUES
  ('task_assigned_to', '88', 'assigned_id',
   NULL, '3c55...02af', :caller_id),
  ('task_assigned_to', '88', 'assigned_id',
   '7a02...b511', NULL, :caller_id);
```

A null `old_value` is an assignment, a null `new_value` is an unassignment.

Response 200 OK returns the updated task with its assignees. Failure cases are those of section 3.3, with 409 covering the in progress to complete transition and 422 covering a null `due_date` or an assignee id that does not resolve.

Additional note on roles. assigning people and completing a task read like admin actions, but a `lab_user` marking their own assigned task complete is the normal way this gets used. Decide whether an assignee may complete their own task, it is the one permission here that is not obvious.